Machine Translated by Google

Classification and Logistic Regression

Machine Translated by Google

Binomial Logistic Regression (1)

- Structure of a generalized linear model

  – Comparison with linear regression model

$$LRM: \ E[Y|x] = \mu_{Y|x} = X\beta = \sum_{i=0}^{m} \beta_i X_i$$

$$GLM: \ E[Y|x] = \mu_{Y|x} = \psi^{-1}(X\beta)$$

$$\psi(\mu) = X\beta$$

ψ : 연결함수(Link Function)

  – If the link function is Logit, the dependent variable is expressed in binary (0 and 1).

  Logistic regression model applicable in capable cases

$$log\left(\frac{p_i}{1-p_i}\right)$$

Machine Translated by Google

Binomial Logistic Regression (2)

- Assuming the dependent variable follows a binomial distribution:

  – Dependent variable to be estimated: (Y=1) probability

$$\psi(\mu) = \log \frac{P(Y=1|X_1, X_2, \cdots, X_p)}{1 - P(Y=1|X_1, X_2, \cdots, X_p)} = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p$$

$$\hat{p}(Y=1|X_1, X_2, \cdots, X_p) = \frac{\exp(\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p)}{1 + \exp(\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p)}$$

(The right-hand side is the sigmoid function)

Machine Translated by Google

Binomial Logistic Regression (3)

- If the composition of the exponential function (exp) is set as a linear expression,

$$log\left(\frac{p_i}{1-p_i}\right) = \beta_0 + \beta_1 x_1$$

– Regression coefficients using simple linear regression analysis ÿ¡It is possible to estimate,

Multiple linear regression analysis is possible even when constructed as a polynomial.

– Critical value for determining 1 and 0 for the estimate of y in the constructed regression model

The (threshold/cut-off value) is commonly set to 0.5, but

All values between (0, 1) are possible.

Machine Translated by Google

Multinomial Logistic Regression Analysis (1)

- Multinomial Logistic Regression

  – The value of the dependent function (y) is not (0, 1) but like (0, 1, , 2, 3, ...), 2 of which are

  It can be utilized when divided into upper classes.

  – Extending the binomial logistic regression model to multinomial logistic regression

  – Multinomial Logistic Regression

  - In Logit Link, it is not the ratio of success-failure odds, but the probability of being a specific class

    Use the ratio of the probability of being of a certain class relative to the rate.

  - If the problem has K classes, estimate (K-1) linear regression equations

Machine Translated by Google

Multinomial Logistic Regression (2)

- Multinomial Logistic Regression
  - In Logit Link, not the ratio of success to failure odds, but the probability of being a specific class versus Use the ratio of probabilities of a certain class
  - If the problem has K classes, estimate (K-1) linear regression equations

$$\ln\frac{P(Y=1)}{P(Y=K)}=X\beta^{(1)},\ \ln\frac{P(Y=2)}{P(Y=K)}=X\beta^{(2)},\ \cdots,\ \ln\frac{P(Y=K-1)}{P(Y=K)}=X\beta^{(K-1)}$$

  - Method to obtain posterior probabilities from the above (K-1) regression equations (composed of sigmoid functions)

$$P(y_i=K)=1\Big/\left(1+\sum_{i=1}^{K-1}\exp\left(\beta^{(i)}X_i\right)\right)$$

$$P(y_i=1)=\exp\left(\beta^{(1)}X_i\right)\Big/\left(1+\sum_{i=1}^{K-1}\exp\left(\beta^{(i)}X_i\right)\right)$$

$$\cdots$$

$$P(y_i=K-1)=\exp\left(\beta^{(K-1)}X_i\right)\Big/\left(1+\sum_{i=1}^{K-1}\exp\left(\beta^{(i)}X_i\right)\right)$$

Machine Translated by Google

Multinomial logistic regression analysis (3)

- Utilization of the softmax function

  – In the equations below, instead of the constant term '1' on the right-hand side ' $\exp\left(\beta^{(K)} X_i\right)$ If you enter ',

$$P(y_i = K) = 1 / \left(1 + \sum_{i=1}^{K-1} \exp\left(\beta^{(i)} X_i\right)\right)$$

$$P(y_i = 1) = \exp\left(\beta^{(1)} X_i\right) / \left(1 + \sum_{i=1}^{K-1} \exp\left(\beta^{(i)} X_i\right)\right)$$

$$\cdots$$

$$P(y_i = K-1) = \exp\left(\beta^{(K-1)} X_i\right) / \left(1 + \sum_{i=1}^{K-1} \exp\left(\beta^{(i)} X_i\right)\right)$$

$$P(y_i = \boldsymbol{j}) = \exp\left(\beta^{(j)} X_i\right) / \sum_{i=1}^{K} \exp\left(\beta^{(i)} X_i\right)$$ (j = *1, … ,K)* * softmax function

  – Using the softmax function instead of the sigmoid function for multiclass problems, multinomial logistic

  It is often used in regression analysis.

Machine Translated by Google

Binomial Logistic Regression Analysis Practice

Machine Translated by Google

Composition of the analysis model (1)

- Data preparation
  - Wisconsin breast cancer dataset
    - Binary classification data determining breast cancer (malignant tumor)
    - Many feature data required for judgment (binary classification)
  - Size, shape, etc. of the tumor
  - Configuration data (data: 569ÿ31, target: 569)

```
import pandas as pd
from sklearn.datasets import load_breast_cancer

b_cancer = load_breast_cancer()
b_cancer_df = pd.DataFrame(b_cancer.data, columns=b_cancer.feature_names)
b_cancer_df['diagnosis'] = b_cancer.target
print('* row number: ', len(b_cancer_df))
print('* column number: ', len(b_cancer_df.columns))
```

```
* row number:  569
* column number:  31
```

Machine Translated by Google

Composition of the analysis model (2)

- Identifying feature information

```
b_cancer_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 31 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   mean radius              569 non-null    float64
 1   mean texture             569 non-null    float64
     ........
 29  worst fractal dimension  569 non-null    float64
 30  diagnosis                569 non-null    int32
dtypes: float64(30), int32(1)
memory usage: 135.7 KB
```

- Identifying and standardizing feature values

```
print(b_cancer.data[0])
from sklearn.preprocessing import StandardScaler  # 데이터 표준화 n(0, 1)
scaler = StandardScaler()
b_cancer_scaled = scaler.fit_transform(b_cancer.data)
print(b_cancer_scaled[0])
```

```
[1.799e+01 1.038e+01 1.228e+02 1.001e+03 1.184e-01 2.776e-01 3.001e-01
 1.471e-01 2.419e-01 7.871e-02 1.095e+00 9.053e-01 8.589e+00 1.534e+02
 6.399e-03 4.904e-02 5.373e-02 1.587e-02 3.003e-02 6.193e-03 2.538e+01
 1.733e+01 1.846e+02 2.019e+03 1.622e-01 6.656e-01 7.119e-01 2.654e-01
 4.601e-01 1.189e-01]
[1.09706398 -2.07333501  1.26993369  0.9843749   1.56846633  3.28351467
 2.65287398  2.53247522  2.21751501  2.25574689  2.48973393 -0.56526506
 2.83303087  2.48757756 -0.21400165  1.31686157  0.72402616  0.66081994
 1.14875667  0.90708308  1.88668963 -1.35929347  2.30360062  2.00123749
 1.30768627  2.61666502  2.10952635  2.29607613  2.75062224  1.93701461]
```

Machine Translated by Google

Composition of the analysis model (2)

- Perform binomial logistic regression analysis using the prepared data
  - Import necessary modules
  - Set 'diagnosis' as Y and b_cancer_scaled scaled by a normal distribution as X.
  - Split the total 569 datasets into training and evaluation data in a 7:3 ratio.
  - Create Logistic Regression Model Object (lr_b_cancer)
  - Train the model using fit() with the training data
  - Predict the trained model using predict() with evaluation data (X_test)

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

Y = b_cancer_df['diagnosis']
X = b_cancer_scaled
X_train, X_test, Y_train, Y_test = train_test_split(X, Y,
                        test_size = 0.3, random_state = 100)
lr_b_cancer = LogisticRegression() # logistic 회귀모델 생성
lr_b_cancer.fit(X_train, Y_train)  # logistic 회귀모델 룬련
Y_predict = lr_b_cancer.predict(X_test) # 테스트 데이터러 예측 수행
```

Machine Translated by Google

Performance evaluation of the configured model

- Verify the performance of the created model – Import necessary modules from the sklearn.metrics package – Calculate misclassification tables for test data split 7:3
  - TP:63, FP:0, FN:6, TN: 102

– Check accuracy (0.965), precision (0.944), recall (1.0), F1 (0.971), etc.

```
from sklearn.metrics import confusion_matrix, accuracy_score
from sklearn.metrics import precision_score, recall_score, f1_score,
                            roc_auc_score

print('* 오차행렬: ', confusion_matrix(Y_test, Y_predict))

accuracy = accuracy_score(Y_test, Y_predict)
precision = precision_score(Y_test, Y_predict)
recall = recall_score(Y_test, Y_predict)
f1 = f1_score(Y_test, Y_predict)
roc_auc = roc_auc_score(Y_test, Y_predict)
print('* 정확도: {0:.3f}, 정밀도: {1:.3f}, 재현율: {2:.3f}, F1: {3:.3f}'.
      format(accuracy, precision, recall, f1))

print('* ROC_AUC: {0:.3f}'.format(roc_auc))
```

```
* 오차행렬:  [[ 63   6]
             [  0 102]]
* 정확도: 0.965, 정밀도: 0.944, 재현율: 1.000, F1: 0.971
* ROC_AUC: 0.957
```

Machine Translated by Google

Multinomial Logistic Regression Analysis Practice

Machine Translated by Google

Data preparation

- iris data
  - 150 samples
  - Input: sepal_length, sepal_width, petal_length, petal_width – target: class (3 classes)

```
from sklearn import datasets
import pandas as pd
iris = datasets.load_iris()

df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['class'] = iris.target

df.columns=['sepal_length', 'sepal_width', 'petal_length', 'petal_width', 'class'] df.head(5)
```

	sepal_length	sepal_width	petal_length	petal_width	class
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

Machine Translated by Google

Training/validation data preparation and one-hot encoding

```
from sklearn.model_selection import train_test_split from
tensorflow.keras.utils import to_categorical

x_data=df.loc[:, 'sepal_length':'petal_width'] # Set input feature as range y_data=df.loc[:,
'class'] # Target feature

# Split 20% of the data sample for validation, and the remaining 80% for
validation x_train, x_valid, y_train, y_valid = train_test_split(x_data, y_data, train_size=0.8, random_state=1)

#one-hot encoding y_train =
to_categorical(y_train) y_valid =
to_categorical(y_valid)

# Print 5 at a
time print(y_train[:5]) ; print(y_valid[:5])
```

```
[[0. 1. 0.]
 [0. 0. 1.]
 [0. 1. 0.]
 [0. 0. 1.]
 [0. 0. 1.]]
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [1. 0. 0.]
 [0. 0. 1.]]
```

Machine Translated by Google

Configuration and Execution of Analysis Model (1)

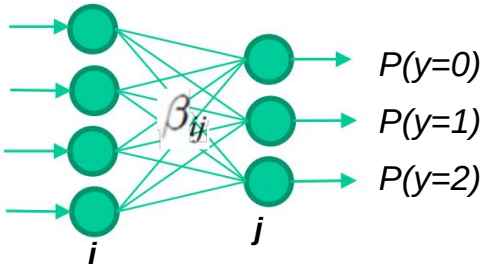
j

- Utilization of artificial neural networks using the Softmax function

  – An artificial neural network with 4 nodes (input features) in the input layer and 3 nodes in the output layer

  – Set the activation function to the softmax function

  – Optimizer (learning rate) setting methods: Adam, SGD, Adagrad, etc.



$$P(y=j)=\exp\left(\beta^{(j)}X_i\right)/\sum_{i=1}^{K}\exp\left(\beta^{(i)}X_i\right)$$

```
# Preparing TensorFlow

from tensorflow.keras import Sequential from
tensorflow.keras.layers import Dense, Activation

# Preparation and configuration of the
neural network model model
= Sequential() model.add(Dense(3, input_dim=4, activation='softmax')) # The number of input variables is 4 and the number of outputs is 3

# Select loss function and optimizer.
model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
```

Machine Translated by Google

Configuration and Implementation of the Analysis Model (2)

- Training: 500 epochs

```
# Repeat training for the number
of epochs hist=model.fit(x_train, y_train, epochs=500, batch_size=20,validation_data=(x_valid, y_valid))
```

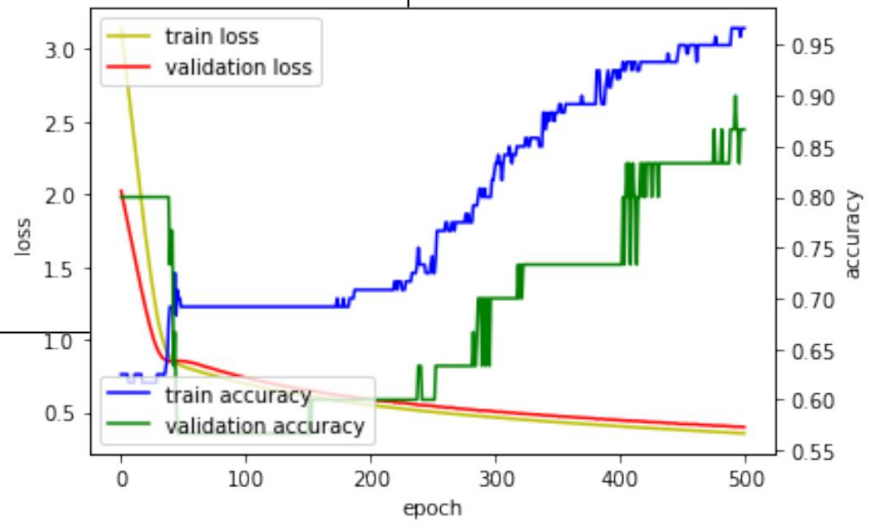
```
Epoch 1/500
6/6 [==========================================] - 0s 11ms/step - loss: 0.3215 - accuracy: 0.9583 - val_loss: 0.3842 - val_accuracy: 0.8333 Epoch 2/500 6/6

[==========================================] - 0s 4ms/step - loss: 0.3216 - accuracy: 0.9583 - val_loss: 0.3804 - val_accuracy: 0.8333 Epoch 3/500 6/6

[==============] - 0s 4ms/step - loss: 0.3207 - accuracy: 0.9583 - val_loss: 0.3792 - val_accuracy: 0.8333

    … …

Epoch 499/500
6/6 [============================] - 0s 4ms/step - loss: 0.1894 - accuracy: 0.9750 - val_loss: 0.2392 - val_accuracy: 0.9667 Epoch 500/500 6/6

[============================] - 0s 4ms/step - loss: 0.1892 - accuracy: 0.9750 - val_loss: 0.2395 - val_accuracy: 0.9333
```

Machine Translated by Google

Configuration and Implementation of the Analysis Model (3)

- Visualization of training process and results

```
# Visualization of the training process

import matplotlib.pyplot as plt fig,
loss_ax = plt.subplots() acc_ax
= loss_ax.twinx()

loss_ax.plot(hist.history['loss'], 'y', label = 'train loss')
loss_ax.plot(hist.history['val_loss'], 'r', label = 'validation loss')

acc_ax.plot(hist.history['accuracy'], 'b', label = 'train accuracy')
acc_ax.plot(hist.history['val_accuracy'], 'g', label = 'validation accuracy')

loss_ax.set_xlabel('epoch')
loss_ax.set_ylabel('loss')
acc_ax.set_ylabel('accuracy')

loss_ax.legend(loc='upper left')
acc_ax.legend(loc='lower left')

plt.show()
```



Machine Translated by Google

Performance evaluation of classification models

Machine Translated by Google

Use of the Misclassification Table (1)

- In classification problems, there are often multiple categories/classes, but they can be divided into two as follows

  Ryuha has many binary classification problems.

  – Customer purchase status in target marketing

  – Good or bad credit status

  – Success or failure, etc.

- Creation of a misclassification table in binary classification

		Predicted		
		TRUE	FALSE	계
Actual	TRUE	O.K. (True Positive)	Missing (False Negative)	AT
	FALSE	False Alarm (False Positive)	O.K. (True Negative)	AF
	계	PT	PF	GS

Machine Translated by Google

Use of the Misclassification Table (2)

- Creation of misclassification tables in binary classification and

Performance metrics

– In binary classification, to the model's prediction results

Confusion Matrix

composition

오분류표 (Confusion Matrix)				
		예측된 Class		합계
		Class = 1	Class = 0	
실제 Class	Class = 1	TP	FN	AT
	Class = 0	FP	TN	AF
합계		PT	PF	GS

- In addition to precision, sensitivity and specificity
  - Sensitivity: Recall of the TRUE class (Recall)
  - Specificity: Recall of the FALSE class (Recall)
- Accuracy using sensitivity and specificity

Calculation possible

– 정밀도 : $Class-specific\ Precision$

$P_P = TP/(TP+FP) = TP/PT$

$P_N = TP/(FN+TN) = TN/PF$

– 민감도 : $Sensitivity\ (True\ Positive\ Rate)$

$TPR = TP/(TP+FN) = TP/AT$

– 특이도 : $Specificity\ (True\ Negative\ Rate)$

$TNR = TN/(FP+TN) = TN/AF$

– 오류율 $= (FP+FN)/GS$, 정확도 $= (TP+TN)/GS$

– 정확도의 다른 계산 :

$Accuracy = Sensitivity \cdot (AT/GS) + Specificity \cdot (AF/GS)$

Machine Translated by Google

Classification performance indicators (1)

- In classification, since the target variable is discrete, the target value and as follows

It can express performance metrics such as error rate and accuracy.

– Goal value (goal variable, label) : $y_i \in C, \ \ C = \{c_1, c_2, \cdots, c_p\}$

– Error rate = $\frac{1}{N}\sum_{i=1}^{N} I(\hat{y}_i \neq y_i)$

- **I(ÿ) is** a function that returns 1 if the condition inside the parentheses is true, otherwise returns 0.

– Accuracy = $\frac{1}{N}\sum_{i=1}^{N} I(\hat{y}_i = y_i) = 1 - \text{오류율}$

Machine Translated by Google

Classification performance indicators (2)

• Calculation of accuracy and error rate as predicted results in multinomial classification –

Representation of classification results in the table below (general cases and examples)

		예측 (Predicted)			
		$\hat{c}_1$	$\hat{c}_2$	$\hat{c}_3$	계
실제(Actual)	c_1	n_{11}	n_{12}	n_{13}	n_1
	c_2	n_{21}	n_{22}	n_{23}	n_2
	c_3	n_{31}	n_{32}	n_{33}	n_3
	계	m_1	m_2	m_3	N

		예측 (Predicted)			
		A	B	C	계
실제(Actual)	A	21	5	1	27
	B	7	53	9	69
	C	3	12	16	31
	계	31	70	26	127

• In the example on the

right above – Number of correctly classified records: 90 (Accuracy:

90/127=0.7087) – Number of misclassified records: 37 (Error rate:

37/127=0.2913) • From the misclassification table on the left above, Precision and Recall used in the fields of pattern recognition or information retrieval

- 클래스별 정밀도 : $\mathrm{Prec}_i = n_{ii}/m_i$,
- 클래스별 재현율 : $Recall_i = n_{ii}/n_i$
- F-척도 : $F_i = 2/\left(\frac{1}{\mathrm{Prec}_i} + \frac{1}{Recall_i}\right) = \frac{2\mathrm{Prec}_i Recall_i}{\mathrm{Prec}_i + Recall_i} = \frac{2n_{ii}}{n_i + m_i}$